

Bootstrap Grid System — Complete Notes

Introduction to the Bootstrap Grid System

What is Bootstrap?

Bootstrap is a popular front-end framework used to develop responsive and mobile-friendly websites quickly.

It provides ready-made CSS classes and JavaScript components that help developers create website layouts without writing all the CSS from scratch.

Bootstrap provides features such as:

- Grid System
- Containers
- Rows and Columns
- Buttons
- Forms
- Navigation Bars
- Cards
- Modals
- Responsive Utilities

What is the Grid System?

The Bootstrap Grid System is a layout system used to divide a webpage into rows and columns.

It helps developers arrange content such as text, images, cards, and forms in an organized manner.

Bootstrap uses a **12-column grid system** by default.

This means that the width of a row can be divided into 12 equal column units.

For example:

- 12 columns = one full-width section.
- 6 columns = half-width section.
- 4 columns = one-third-width section.
- 3 columns = one-fourth-width section.
- 2 columns = one-sixth-width section.

Why Do We Need the Grid System?

Suppose we want to create a webpage containing three student profile cards in one row.

Without a grid system, we would need to manually calculate widths, margins, and spacing.

With Bootstrap, we can use predefined classes to arrange the cards.

Example:

```
<div class="container">
  <div class="row">
    <div class="col-4">Student 1</div>
    <div class="col-4">Student 2</div>
    <div class="col-4">Student 3</div>
  </div>
</div>
```

Each column occupies 4 out of 12 grid units.

Therefore:

$$4 + 4 + 4 = 12$$

All three columns fit in one row.

Advantages of the Bootstrap Grid System

1. Makes webpage layouts easier to create.
2. Supports responsive website design.
3. Reduces the need to write custom CSS for common layouts.
4. Provides consistent spacing and alignment.
5. Supports different layouts for mobile, tablet, and desktop screens.
6. Allows columns to be nested inside other columns.
7. Makes it easier to rearrange content for different screen sizes.

Understanding the 12-Column System

Bootstrap divides a row into 12 equal grid units.

Each column class specifies how many of those units the element should occupy.

Common Column Sizes

Bootstrap Class	Grid Units	Approximate Width
col-12	12	100%
col-11	11	91.67%
col-10	10	83.33%
col-9	9	75%
col-8	8	66.67%
col-7	7	58.33%
col-6	6	50%
col-5	5	41.67%
col-4	4	33.33%
col-3	3	25%
col-2	2	16.67%
col-1	1	8.33%

These percentages represent the approximate share of the row's available width occupied by each column, excluding the effects of gutters and other layout details.

Understanding Column Combinations

Different column sizes can be combined as long as their total is 12 or less.

Examples:

Example 1: Two equal columns

```
<div class="row">  
  <div class="col-6">Left Side</div>  
  <div class="col-6">Right Side</div>  
</div>
```

Calculation:

$$6 + 6 = 12$$

Each column occupies half of the row.

Example 2: Three equal columns

```
<div class="row">  
  <div class="col-4">Column 1</div>  
  <div class="col-4">Column 2</div>  
  <div class="col-4">Column 3</div>  
</div>
```

Calculation:

$$4 + 4 + 4 = 12$$

Each column occupies approximately one-third of the row.

Example 3: Four equal columns

```
<div class="row">
  <div class="col-3">Column 1</div>
  <div class="col-3">Column 2</div>
  <div class="col-3">Column 3</div>
  <div class="col-3">Column 4</div>
</div>
```

Calculation:

$$3 + 3 + 3 + 3 = 12$$

Each column occupies one-fourth of the row.

Example 4: Unequal columns

```
<div class="row">
  <div class="col-8">Main Content</div>
  <div class="col-4">Sidebar</div>
</div>
```

Calculation:

$$8 + 4 = 12$$

The main content occupies two-thirds of the row, while the sidebar occupies one-third.

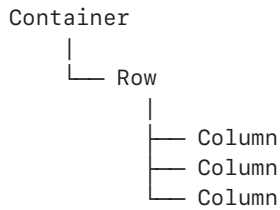
Important Rule: If the total column units exceed 12, the extra columns may wrap onto another line, depending on the layout and available space.

The Three Main Grid Components

The Bootstrap grid system mainly uses three components:

1. Container
2. Row
3. Column

The usual structure is:



Container

A container is used to hold and align the main content of a webpage.

Bootstrap provides two commonly used container classes:

- `.container`
- `.container-fluid`

A. `.container`

The `.container` class provides a responsive, centered layout with a maximum width that changes at Bootstrap's responsive breakpoints.

Example:

```
<div class="container">
  <h1>Welcome to My Website</h1>
  <p>This content is inside a container.</p>
</div>
```

Use `.container` when you want content to remain centered and constrained in width on larger screens.

B. `.container-fluid`

The `.container-fluid` class occupies the full available width of its parent, usually the viewport.

Example:

```
<div class="container-fluid">
  <h1>Full Width Layout</h1>
  <p>This content uses the available width.</p>
</div>
```

Use `.container-fluid` when you want a full-width layout.

Difference Between `.container` and `.container-fluid`

Feature	.container	.container-fluid
Width	Responsive maximum width	100% of available width
Centered layout	Yes, with automatic horizontal margins	Usually fills its parent
Common use	Main page content	Full-width sections and layouts
Responsive	Yes	Yes

Row

A row is used to group columns horizontally.

Example:

```
<div class="container">
  <div class="row">
    <div class="col-6">First Column</div>
    <div class="col-6">Second Column</div>
  </div>
</div>
```

Explanation:

- `.container` holds the layout.
- `.row` groups the columns.
- Each `.col-6` occupies half the row.

A row also manages the spacing between columns through Bootstrap's gutter system.

Important: In the standard Bootstrap grid, columns should generally be placed inside rows, and rows should be placed inside containers or another suitable grid column.

Column

A column holds the actual content of the grid.

It may contain text, images, buttons, cards, forms, or even another row.

Example:

```
<div class="container">
  <div class="row">
    <div class="col-4">
      <h2>HTML</h2>
      <p>Learn HTML basics.</p>
    </div>

    <div class="col-4">
      <h2>CSS</h2>
      <p>Learn styling.</p>
    </div>

    <div class="col-4">
      <h2>Bootstrap</h2>
      <p>Learn responsive layouts.</p>
    </div>
  </div>
</div>
```

Each column contains a heading and a paragraph.

Equal-Width Columns

Bootstrap can automatically distribute available space among columns when you use `.col` without a number.

Example:

```
<div class="container">
  <div class="row">
    <div class="col">Column 1</div>
    <div class="col">Column 2</div>
    <div class="col">Column 3</div>
  </div>
</div>
```

Bootstrap distributes the available row width equally among the three columns.

When Should We Use `.col`?

Use `.col` when:

- All columns should share the available space equally.
- You do not need to specify exact grid-unit sizes.
- You want Bootstrap to distribute the available width automatically.

Difference Between `.col` and `.col-4`

Class	Meaning
.col	Automatically shares available width
.col-4	Occupies 4 of the 12 grid units
.col-6	Occupies 6 of the 12 grid units
.col-12	Occupies all 12 grid units

Example:

```
<div class="row">
  <div class="col">A</div>
  <div class="col">B</div>
</div>
```

Both columns share the available space equally.

```
<div class="row">
  <div class="col-4">A</div>
  <div class="col-8">B</div>
</div>
```

The first column occupies one-third of the row, and the second occupies two-thirds.

6. Responsive Design

What Is Responsive Design?

Responsive design is an approach to webpage development in which the layout adapts to different screen sizes.

A responsive website should be usable on:

- Mobile phones
- Tablets
- Laptops
- Desktop computers
- Large displays

For example, three cards may appear side by side on a desktop but stack vertically on a mobile device.

Bootstrap makes responsive design easier through its grid classes and breakpoints.

Why Is Responsive Design Necessary?

Different devices have different screen widths.

A layout that looks good on a desktop may become difficult to read on a mobile phone if all its columns remain side by side.

Responsive layouts help make content easier to read and interact with across devices.

Bootstrap Breakpoints

What Is a Breakpoint?

A breakpoint is a screen-width threshold at which a responsive layout or style can change.

Bootstrap 5.3 provides six default grid breakpoints.

Breakpoint	Class Infix	Minimum Viewport Width	Typical Device Category
Extra small	None	Below 576px	Small phones
Small	sm	576px	Larger phones
Medium	md	768px	Tablets
Large	lg	992px	Laptops and desktops
Extra large	xl	1200px	Large desktops
Extra extra large	xxl	1400px	Very large desktops

Important: These widths are Bootstrap's default breakpoint thresholds, not strict device classifications. Actual device widths vary.

Bootstrap follows a mobile-first approach. This means that classes without breakpoint prefixes generally apply to all screen sizes unless overridden by a more specific responsive class.

What Does Mobile-First Mean?

Mobile-first means that we begin with styles suitable for smaller screens and then add responsive changes for larger screens.

For example:

```
<div class="col-12 col-md-6">  
  Content  
</div>
```

This means:

- Below the `md` breakpoint, the column occupies the full row.
- At `md` and above, the column occupies half the row.

Responsive Column Classes

Responsive column classes allow us to control column widths at different breakpoints.

The general syntax is:

```
col-{breakpoint}-{size}
```

Examples:

- `col-sm-6`
- `col-md-4`
- `col-lg-3`
- `col-xl-2`
- `col-xxl-1`

The breakpoint prefix specifies the minimum viewport width at which the declared grid size applies.

Example: Full Width on Mobile, Half Width on Medium Screens

```
<div class="container">
  <div class="row">
    <div class="col-12 col-md-6">
      Column 1
    </div>

    <div class="col-12 col-md-6">
      Column 2
    </div>
  </div>
</div>
```

Explanation:

- `col-12` makes each column full width by default.
- `col-md-6` changes each column to half width at 768px and above.

Expected layout:

Mobile:

```
+-----+
|      Column 1      |
+-----+
|      Column 2      |
+-----+
```

Medium screens and above:

```
+-----+-----+
| Column 1 | Column 2 |
+-----+-----+
```

Example : Three Columns on Desktop, One Column on Mobile

```
<div class="container">
  <div class="row">
    <div class="col-12 col-lg-4">HTML</div>
    <div class="col-12 col-lg-4">CSS</div>
    <div class="col-12 col-lg-4">Bootstrap</div>
  </div>
</div>
```

Explanation:

- On small screens, each element occupies the full row.
- At 992px and above, each occupies 4 grid units.
- The three columns fit into one row on large screens.

Example : Different Widths at Multiple Breakpoints

```
<div class="container">
  <div class="row">
    <div class="col-12 col-sm-6 col-lg-3">
      Item 1
    </div>

    <div class="col-12 col-sm-6 col-lg-3">
      Item 2
    </div>

    <div class="col-12 col-sm-6 col-lg-3">
      Item 3
    </div>

    <div class="col-12 col-sm-6 col-lg-3">
      Item 4
    </div>
  </div>
</div>
```

Expected layout:

- Below 576px: one item per row.
- At 576px and above: two items per row.
- At 992px and above: four items per row.

Important Rule About Breakpoint Classes

Responsive classes generally apply at their specified breakpoint and larger breakpoints until another applicable class overrides the size.

For example:

```
<div class="col-12 col-md-6 col-lg-4">
  Responsive Column
</div>
```

This means:

- Extra-small and small screens: 12 grid units.
- Medium screens: 6 grid units.
- Large and larger screens: 4 grid units.

. Auto Layout Columns

Auto layout columns use `.col` to distribute available space without explicitly setting a grid size.

Example:

```
<div class="container">
  <div class="row">
    <div class="col">First</div>
    <div class="col">Second</div>
    <div class="col">Third</div>
  </div>
</div>
```

All three columns receive equal shares of the available row width.

Combining Fixed and Automatic Columns

Example:

```
<div class="container">
  <div class="row">
    <div class="col">Flexible Column</div>
    <div class="col-6">Half Width</div>
    <div class="col">Flexible Column</div>
  </div>
</div>
```

The `.col-6` column occupies half the row. The two `.col` columns share the remaining available space.

Auto-layout behavior can depend on the combination of column classes and the available width.

Gutters: Spacing Between Columns

What Is a Gutter?

A gutter is the spacing between columns in the Bootstrap grid.

Bootstrap uses horizontal and vertical gutter spacing to keep content from appearing crowded.

Gutters are controlled by gutter utility classes.

Horizontal Gutters: `.gx-*`

The `.gx-*` classes control horizontal gutter spacing.

Examples:

- `.gx-0` — no horizontal gutter.
- `.gx-1` — small horizontal gutter.
- `.gx-3` — standard-sized horizontal gutter.
- `.gx-5` — larger horizontal gutter.

Example:

```
<div class="container">
  <div class="row gx-5">
    <div class="col-6">
      Column 1
    </div>

    <div class="col-6">
      Column 2
    </div>
  </div>
</div>
```

Vertical Gutters: `.gy-*`

The `.gy-*` classes control vertical gutter spacing between rows of columns.

Example:

```
<div class="container">
  <div class="row gy-4">
    <div class="col-6">Item 1</div>
    <div class="col-6">Item 2</div>
    <div class="col-6">Item 3</div>
    <div class="col-6">Item 4</div>
  </div>
</div>
```

The columns wrap into multiple rows, and vertical gutters provide spacing between the rows.

Both Horizontal and Vertical Gutters: `.g-*`

The `.g-*` classes control both horizontal and vertical gutters.

Example:

```
<div class="container">
  <div class="row g-3">
    <div class="col-6">Item 1</div>
    <div class="col-6">Item 2</div>
    <div class="col-6">Item 3</div>
    <div class="col-6">Item 4</div>
  </div>
</div>
```

Common Gutter Classes

Class	Purpose
g-0	No horizontal or vertical gutter
g-1	Small gutters in both directions
g-2	Slightly larger gutters
g-3	Medium gutters
g-4	Large gutters
g-5	Larger gutters
gx-3	Horizontal gutters only
gy-3	Vertical gutters only

Bootstrap's default gutter spacing scale is based on its spacing utilities, and the actual visual result depends on the selected value and surrounding layout.

Responsive Grid Utilities

Bootstrap provides responsive utilities that work alongside the grid system.

These utilities can control visibility, spacing, alignment, and other layout properties at different screen sizes.

Responsive Display Utilities

Common display classes include:

- `.d-none` — hides an element.
- `.d-block` — displays an element as a block.
- `.d-flex` — displays an element as a flex container.
- `.d-inline` — displays an element as inline content.
- `.d-inline-block` — displays an element as inline-block content.

Example:

```
<div class="d-none d-md-block">  
  Visible on medium screens and larger.  
</div>
```

The element is hidden below the `md` breakpoint and displayed as a block at `md` and above.

Responsive Spacing Utilities

Bootstrap spacing utilities can apply margins and padding.

Examples:

- `.mt-3` — top margin.

- `.mb-3` — bottom margin.
- `.ms-3` — start-side margin.
- `.me-3` — end-side margin.
- `.p-3` — padding on all sides.
- `.px-3` — horizontal padding.
- `.py-3` — vertical padding.

Responsive variants are also available.

Example:

```
<div class="p-2 p-md-4">  
  Responsive Padding  
</div>
```

The element uses smaller padding by default and larger padding at the `md` breakpoint and above.

Important: Spacing utilities are separate from grid sizing classes. They can be combined with grid classes to improve the layout.